Robustness of Policy-Gradient Reinforcement Learning for Multi-Echelon Inventory Control: A Cross-Environment Study on Stationary and Non-Stationary Supply Chains

Divyank Singh, Ishan Biswas
Khoury College of Computer Sciences, Northeastern University
CS 5180 – Reinforcement Learning and Sequential Decision Making
singh.divya@northeastern.edu, biswas.is@northeastern.edu

Abstract

Recent papers report that deep reinforcement-learning policies beat classical Operations Research heuristics on multi-echelon inventory control by $10\,\%$ to $40\,\%$. Almost all of those numbers come from a single hand-crafted environment and a fixed baseline, which leaves the obvious question unanswered: is the RL agent really better, or is the baseline simply a straw man on that particular instance? We take the two policy-gradient agents we built for this course project (PPO on a 1-warehouse/3-retailer divergent chain, A3C on a 4-node factory–middle–leaf MEIS) and run each of them on two environments that share the same observation and action spaces: the stationary benchmark we started from (Env-1) and a harder non-stationary variant (Env-2) with seasonal demand, cross-retailer correlation, stochastic lead times, supplier capacity caps and partial lost sales. Against the analytical base-stock baseline, PPO's win jumps from $36.76\,\%$ on Env-1 to $94.20\,\%$ on Env-2; the extra margin mostly comes from the untuned baseline falling apart under shocks, not from PPO improving. Against an (s, S) baseline that is retuned per environment, A3C wins by $1.64\,\%$ on Env-1 and then loses by $21.79\,\%$ on Env-2. Our takeaway: a single-environment win is not, on its own, evidence that the RL policy is better, and any deep-RL inventory result should come with at least one harder distributional variant and an equivalently-tuned baseline.

Code — https://github.com/singhdivyank/multi-echelon-rl-inventory

1 Introduction

Supply-chain operations are one of the larger line items on any manufacturing firm's books. Inventory alone makes up $20\,\%$ to $60\,\%$ of total assets for a typical manufacturer (Unyimadu and Anyibuofu 2014), and inventory management is the part of the business that has to pick how much stock to keep at each tier so demand gets served at the lowest cost. Written down, the objective is just

$$\min\ \mathbb{E}\left[\sum_{t=0}^{T-1}\left(C_t^{\text{hold}} + C_t^{\text{back}} + C_t^{\text{ord}}\right)\right], \tag{1}$$

where C^{hold} is the cost of storing unsold inventory, C^{back} is what you pay when demand walks in and cannot be served, and C^{ord} is the fixed-plus-variable cost of placing a replenishment order. A Multi-Echelon Inventory System (MEIS) solves this jointly across suppliers, manufacturers, warehouses and retailers, instead of one tier at a time. That joint view is where the problem gets hard: a decision that looks sensible locally at one echelon can blow up cost a tier over.

Classical Operations Research has two textbook answers to this. The echelon base-stock policy and the (s, S) reorder-point policy are both optimal under strict stationarity and i.i.d. demand (with fixed ordering cost in the (s, S) case) (Clark and Scarf 1960). Meta-heuristic extensions, e.g. genetic algorithms (Grahl et al. 2016), do a decent job on richer instances, but they carry two familiar caveats: no optimality guarantee, and a tendency to degrade whenever the underlying dynamics change. Once demand becomes non-stationary, shocks appear or lead times grow heavy-tailed, both classes start to misfire, because their set-points are tuned against the *mean* demand-over-lead-time and nothing else.

Deep reinforcement learning (Sutton and Barto 2018) is appealing here for exactly that reason: a learned policy can, at least in principle, adapt to whatever dynamics the MDP happens to express. Recent papers (Oroojlooyjadid et al. 2022; Gijsbrechts et al. 2022; Geevers et al. 2024; Hammler et al. 2022) report substantial wins over OR heuristics, often in the $10\,\%$ to $40\,\%$ range. Those comparisons tend to share a weakness, though: a single hand-crafted environment, and a baseline whose parameters are never re-tuned for that environment. It is then hard to tell how much of the gap is the RL agent actually being better versus the baseline being handicapped on a problem it would otherwise solve well.

We started this project with a more modest setup: one policy-gradient agent per algorithm (PPO, A3C) on a single environment each (a divergent supply chain for PPO, a 4-node MEIS chain for A3C). A reviewer at our poster session flagged exactly the concern above and asked whether the wins survived in a harder setting. The present paper is our answer. On top of the baseline story, three specific choices make the comparison informative:

1. *Continuous-action PPO.* The PPO agent uses a fully continuous 4-dimensional action head (one normalised order quantity per echelon), rather than the discretised grids that most of the prior literature uses.
2. *Explicit backorder accounting.* Both environments carry

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

unfulfilled demand as an ageing backlog and fold it back into the cost signal, so the RL agent sees the same shortage penalty that classical OR treats as a first-class citizen.

3. *A shared distribution-shift harness.* We build a non-stationary second environment (Env-2) per benchmark with seasonal demand, cross-retailer correlation, rare multiplicative shocks, heavy-tailed lead times, stochastic capacity caps and partial lost sales, but with exactly the same observation and action spaces as Env-1. The agent code transfers without any change, and the Env-1 hyperparameters carry over to Env-2 verbatim, so whatever cost differences we read off reflect dynamics rather than tuning effort.

With that harness in place, the results split cleanly into one positive and one negative case. PPO's margin over its fixed analytical baseline gets *wider* on Env-2, while A3C's narrow Env-1 win over a retuned (s, S) baseline flips into a decisive loss once the environment becomes harder. Neither result on its own tells the whole story: the sign of a deep-RL-vs-heuristic comparison turns out to depend quite strongly on both the environment and how seriously the baseline is tuned. The rest of the paper is laid out as follows. Section 2 sets up the MDP, the two DRL algorithms and the OR baselines; Section 3 places the work in the literature; Section 4 gives the two environments formally; Section 5 walks through the four evaluation cells; Section 6 wraps up.

2 Background

2.1 The Multi-Echelon Inventory MDP

Both benchmarks are cast as a finite-horizon Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, P, C, \gamma, T \rangle$. The state $s_t \in \mathcal{S}$ packs on-hand inventory, in-transit pipeline stock, and backlog at every non-factory node; the action $a_t \in \mathcal{A}$ is a per-node order quantity (a real vector for PPO, a 13-way discrete choice for A3C). The per-step cost $C(s_t, a_t) \geq 0$ sums the holding, backorder and ordering terms, plus lost-sales penalties on Env-2. A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ is chosen to minimise the expected discounted cost

$$J(\pi) \;=\; \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t \, C(s_t, a_t) \right]. \tag{2}$$

Since the RL agent works with returns, we let it maximise $R(\pi) = -J(\pi)$. The resulting reward signal is dense, bounded below by 0 and unbounded above.

2.2 Classical OR Heuristics

Echelon Base-Stock (PPO Baseline) Each echelon i is assigned a base-stock level S_i and, at every step, orders enough to top its inventory position back up to S_i:

$$a_{i,t} \;=\; \max(0,\; S_i - \mathrm{IP}_{i,t}), \tag{3}$$

with $\mathrm{IP}_{i,t}$ = on-hand + in-transit − backlog at node i. The S_i values come from the Env-1 demand-over-lead-time distribution in closed form and are never re-tuned afterwards; the same numbers are reused on Env-2. This is deliberate, and it is how base-stock baselines are usually reported in the literature.

(s, S) Reorder-Point Policy (A3C Baseline) Each node i carries both a reorder point s_i and an order-up-to level S_i: whenever $\mathrm{IP}_{i,t} \leq s_i$, we order $S_i - \mathrm{IP}_{i,t}$ units, otherwise nothing. Unlike the base-stock baseline, (s_i, S_i) is tuned separately for each environment via differential evolution (100 generations, 30 Monte-Carlo episodes per candidate), so both the Env-1 and Env-2 (s, S) baselines are near-optimal under their own dynamics. That asymmetry (a fixed base-stock vs. a retuned (s, S)) will come back in Section 5 and matters for how the headline numbers should be read.

2.3 Policy-Gradient RL

PPO The PPO side uses the clipped-surrogate variant (Schulman et al. 2017). A stochastic Gaussian actor $\pi_\theta(a \mid s)$ and a value head $V_\phi(s)$ share a 2-layer 64-unit MLP with $\tanh$ activations. We collect rollouts of length B=512 under $\pi_{\theta_{\text{old}}}$, compute Generalized Advantage Estimates $\hat{A}_t$ (Schulman et al. 2016), and then update θ for 5 epochs of size-64 minibatches with the clipped objective

$$\begin{aligned} \mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \Big[\min\big(& r_t(\theta)\, \hat{A}_t, \\ & \mathrm{clip}(r_t(\theta), 1{-}\varepsilon, 1{+}\varepsilon)\, \hat{A}_t \big) \Big], \end{aligned} \tag{4}$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ and ε=0.2. On top of the clipped surrogate we add a squared-error value loss and an entropy bonus so the policy does not go deterministic too early:

$$\mathcal{L}^{\text{VF}} = \mathbb{E}_t\big[(V_\phi(s_t) - R_t)^2\big], \;\; \mathcal{L}^{\text{ENT}} = \mathbb{E}_t\big[\mathcal{H}[\pi_\theta(\cdot \mid s_t)]\big], \tag{5}$$

$$\mathcal{L}(\theta, \phi) = \mathcal{L}^{\text{CLIP}} - c_1\, \mathcal{L}^{\text{VF}} + c_2\, \mathcal{L}^{\text{ENT}}. \tag{6}$$

The $\hat{A}_t$ themselves come from GAE, which chains single-step TD residuals $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ into the low-variance multi-step estimate $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$.

A3C The original A3C (Mnih et al. 2016) runs several actor-learners in parallel, each with its own copy of the environment, and lets them push asynchronous gradient updates into a shared set of weights, which is where the name comes from. Our MEIS episodes are short (365 steps) and environment simulation, not gradient compute, is what eats the wall-clock, so we run a synchronous single-worker variant instead. Learning dynamics are effectively the same and the cross-process bookkeeping goes away. The actor emits a softmax over $|\mathcal{A}|$=13 actions, the critic emits a scalar value, and we use an n-step-advantage policy gradient:

$$\begin{aligned} \nabla_\theta J \approx\; & \mathbb{E}_t\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\big] \\ & +\; \beta\, \nabla_\theta \mathcal{H}[\pi_\theta(\cdot \mid s_t)]. \end{aligned} \tag{7}$$

The entropy term $\mathcal{H}[\pi_\theta(\cdot \mid s_t)]$, weighted by β, is a standard regulariser: it keeps the action distribution from collapsing onto a bad local optimum too early in training.

2.4 Significance Testing

Each evaluation cell gives us hundreds of per-episode costs, so for the RL-vs-baseline comparison we report (i) mean ± one standard deviation and (ii) a 95% confidence interval on the mean.

3 Related Work

Classical and Meta-Heuristic Methods The Clark–Scarf decomposition (Clark and Scarf 1960) is the reference point for tractable multi-echelon policies: it proves optimality of echelon base-stock when demand is strictly stationary, i.i.d. and backlog dynamics are well-behaved. Meta-heuristics like the genetic algorithm of Grahl et al. (2016) push that further into richer instances with differentiated service times and non-trivial topologies, at the cost of giving up the optimality guarantee and a fair bit of robustness to structural change. Our own baselines, the analytical echelon base-stock (PPO side) and the tuned (s, S) (A3C side), are direct descendants of these two lines; we treat them as targets to beat rather than methods we are studying in their own right.

Deep RL for Multi-Echelon Inventory Geevers et al. (2024) train PPO on a divergent MEIS with stochastic demand and benchmark it against an (s, S) heuristic. Our PPO pipeline follows their problem formulation closely, with two deliberate changes: we keep the action head fully continuous rather than discretising it, and we hold back an explicit non-stationary Env-2 for out-of-distribution evaluation. Hammler et al. (2022) report an A3C variant on a similar MEIS problem, which is where our 13-way discretised action head comes from; we keep the architecture but, again, evaluate the trained policy on a harder Env-2 instead of stopping at the stationary single environment. Lu et al. (2025) train deep-RL policies on supply chains with explicitly *disruptive* dynamics (supplier shocks, lead-time spikes) and see double-digit gains over classical baselines. That line of work is useful background for our PPO result: the stressors Env-2 injects are exactly the ones classical OR tends to misfire on, which is most of the reason PPO's *relative* advantage widens rather than shrinks.

Value-Based RL for Inventory Oroojlooyjadid et al. (2022) apply deep Q-learning to the beer game. We could have swapped A3C for DQN on the MEIS benchmark since the action space is already discrete; we did not, because DQN tends to be sample-inefficient on long-horizon inventory rollouts with dense rewards.

Robustness Benchmarks Outside Inventory Procgen (Cobbe et al. 2020) makes the same point we make, just outside inventory control: agents that look strong on a hand-crafted level often collapse on procedurally-sampled levels that share the observation and action spaces. Env-2 plays the role of that "procedural" counterpart to Env-1, specialised to an OR-flavoured inventory setting.

4 Project Description

This section writes out, in formal detail, the two environments, the two agents and their baselines, and the training protocol.

4.1 Environment 1 (Stationary)

PPO / Divergent One warehouse feeds three retailers. Each retailer i sees i.i.d. Poisson demand with rate $\lambda_i \sim \mathrm{U}[5, 15]$ per day, lead time is a deterministic single step, and there are no capacity caps. Anything that cannot be served is backlogged (i.e. appears as negative inventory). The observation has 14 components: warehouse inventory and buffer, three in-transit pipelines, three retailer inventories, three retailer backorders, one outstanding warehouse order and a normalised step index. The action is a 4-dimensional vector of normalised order quantities, one per echelon, and episodes run for $T{=}500$ steps.

Table 1: Comparison of experimental environments.

Feature	Baseline (Env-1)	Complex (Env-2)
Demand profile	Stationary (Poisson)	Non-stationary (seasonal / trend)
Demand correlation	Independent across nodes	Correlated (shared latent factor)
Supply-chain capacity	Unconstrained	Bottleneck-limited (μ=12,000)
Lead-time dynamics	Deterministic / stable	Stochastic (spikes / variability)
Penalty structure	Standard backlog costs	Penalty-based lost sales

A3C / MEIS This is a 4-node divergent multi-echelon inventory system: one factory with infinite capacity, one middle warehouse, and two leaf retailers. Leaf demand is Gaussian $\mathcal{N}(3300, 100)$; lead times are Gaussian $\mathcal{N}(\mu{=}2, \sigma{=}1)$ rounded to integer days. Backlog ages out after 7 days, i.e. demand unserved for longer than a week is treated as lost and incurs a shortage penalty. The action space is discrete, with 13 options ("do nothing" plus $4{\times}3{=}12$ pre-set reorder combinations), and episodes run for $T{=}365$ steps.

4.2 Environment 2 (Non-Stationary Complex)

Env-2 keeps the Env-1 observation and action spaces intact and only changes the transition dynamics and reward. Table 1 lays the two environments side by side; the concrete numerics (amplitudes, probabilities, noise standard deviations) live in the YAML configs shipped with the code.

4.3 Algorithms and Baselines

PPO on the continuous-action divergent environment is trained with the clipped-surrogate objective (Eq. 4) against a fixed base-stock baseline (Eq. 3). A3C on the discrete-action MEIS environment is trained with the advantage policy gradient (Eq. 7) against a tuned (s, S) baseline.

5 Experiments

5.1 Why These Experiments

We ran the four cells $\{\mathrm{PPO}, \mathrm{A3C}\} \times \{\text{Env-1}, \text{Env-2}\}$ with seed 42 and the hyperparameters in Table 2. The minimum the reviewer's question asks for is the four paired RL-vs-baseline comparisons; on top of that we include training curves so the convergence behaviour is visible too.

5.2 Headline Numbers

Table 3 gives the main result. Δ is the relative mean-cost improvement of RL over its heuristic baseline, with positive

Table 2: Selected hyperparameters (identical across Env-1 / Env-2 per algorithm).

Hyperparameter	PPO	A3C
Iters / episodes	5,000	10,000
Hidden layers $\times$ width	2×64 (tanh)	3×64 (ReLU)
Discount γ	0.95	0.99
GAE λ	0.95	0.95
Clip ε	0.2	n/a
Learning rate	$3{\times}10^{-4}$	$1{\times}10^{-4}$
Rollout / batch	512 / 64	per-episode
Entropy coef. β	$5{\times}10^{-3}$	10^{-3}
Value-loss coef.	0.25	0.5
Grad. clip $\|\nabla\|$	0.5	0.5
Eval episodes	100	1000
Seed	42	42

numbers meaning RL wins. Service level is only exposed by the MEIS environment, which is why the PPO rows read "n/a".

5.3 Training Dynamics

Figure 1 and Figure 2 show per-episode training cost (light) and a rolling mean (bold) for each cell.

5.4 PPO: When the Win Grows on the Harder Environment

Figure 3 plots Table 3 as grouped bar charts. On Env-1, PPO beats the fixed analytical base-stock by $36.76\,\%$; on Env-2 that margin widens to $94.20\,\%$. What is going on is not that PPO gets better in absolute terms. The absolute cost on Env-2 sits on a different scale anyway, because Env-2 adds lost-sales penalties and widens some state bounds. The widening is really the untuned base-stock baseline falling over under capacity caps and shocks. Figure 4 shows the flip side: PPO's episode-cost distribution has a pronounced right tail ($\sigma/\mu \approx 1.60$). A handful of episodes hit shock sequences that the agent has not learned to hedge against, and those episodes sit an order of magnitude above the mean.

When PPO Succeeds On both environments, provided the baseline is the fixed analytical one. Two things help. The baseline's set-points are built purely from the mean demand-over-lead-time, whereas PPO's policy can condition on the full state, including current inventory and in-transit pipeline. And on Env-2, those set-points are not even calibrated against the right mean any more (seasonality and shocks make sure of that), which is what lets PPO's relative advantage grow.

When PPO Would Fail Against a retuned base-stock (which we do not report here) the margin would shrink substantially. And even against the fixed baseline, a CVaR-weighted evaluation would flag PPO's Env-2 right tail as an unresolved risk that the mean hides.

5.5 A3C: When the Env-1 Win Does Not Transfer

On Env-1, A3C edges past a retuned (s, S) policy by a narrow $1.64\,\%$. On Env-2 the sign flips: A3C is now $21.79\,\%$ worse than the retuned baseline. Figure 5 makes the effect size easy to see. The (s, S) cost distribution sits almost entirely to the left of A3C's, which is stochastic dominance in favour of the heuristic. Figure 6 shows the same picture on service level.

When A3C Succeeds On Env-1 only, and only just. Stationary demand is already approximated well by a single (s, S) pair, but A3C's 13-way action head still finds a few slightly-better-than-heuristic moments to place orders, because it reacts to the full pipeline state rather than inventory position alone.

When A3C Fails, and Why Env-2 breaks this, and three things compound:

1. *Discretised action head.* A3C picks from 13 fixed order combinations. Under seasonal demand the optimal per-node order moves smoothly, and a coarse 4-level grid just cannot match that shape.
2. *No non-stationarity representation.* The policy carries no recurrent state, no episode-time feature and no shock or season indicator. Any response to non-stationarity has to come from the training distribution, and 10,000 episodes are not enough to bake it in.
3. *The baseline is retuned, A3C is not.* The (s, S) tuner runs differential evolution directly on Env-2's dynamics, while A3C is stuck with its Env-1 hyperparameters. The comparison is fair in the sense that it matches standard practice, but it is asymmetric in how much effort each side gets to spend.

Fixing any one of these in isolation would probably close much of the gap.

6 Conclusion

6.1 Summary of Results

We trained one PPO agent on a divergent inventory environment and one A3C agent on a 4-node MEIS environment under stationary conditions (Env-1), and then re-trained both under identical hyperparameters on a harder, non-stationary variant (Env-2) that shares the Env-1 observation and action spaces. Against a fixed analytical base-stock baseline, PPO's cost improvement moved from $36.76\,\%$ on Env-1 to $94.20\,\%$ on Env-2. Against a retuned (s, S) baseline, A3C went from $+1.64\,\%$ on Env-1 to $-21.79\,\%$ on Env-2.

6.2 What We Learned

Two takeaways from the four cells, in rough order of generality:

1. *Single-environment wins are not robust evidence.* The same PPO training that produces a $36.76\,\%$ win on Env-1 produces a $94.20\,\%$ win on Env-2, but most of that difference comes from the baseline degrading, not from the RL agent improving. On the A3C side, the narrow Env-1 win turns into a decisive Env-2 loss as soon as the baseline is allowed to retune itself. Reporting a single number, on one environment, is just not enough.

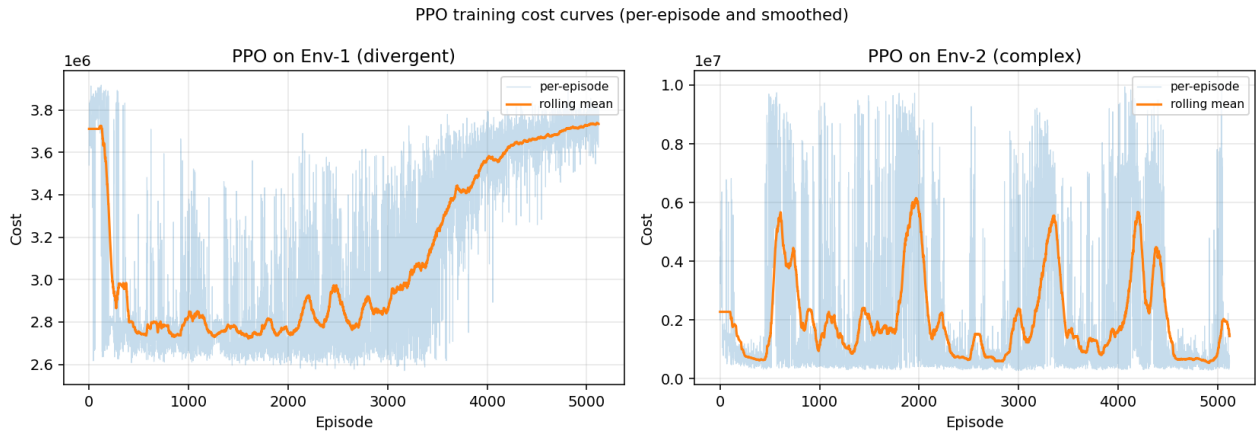

Figure 1: PPO training cost on Env-1 (left) and Env-2 (right). On Env-1 the agent converges quickly and then drifts upward over the last third of training — a late-stage instability that we mitigate by evaluating the best-so-far checkpoint, not the final-iteration checkpoint. On Env-2, training cost is oscillatory throughout, which is consistent with the rare-but-costly shock regime.

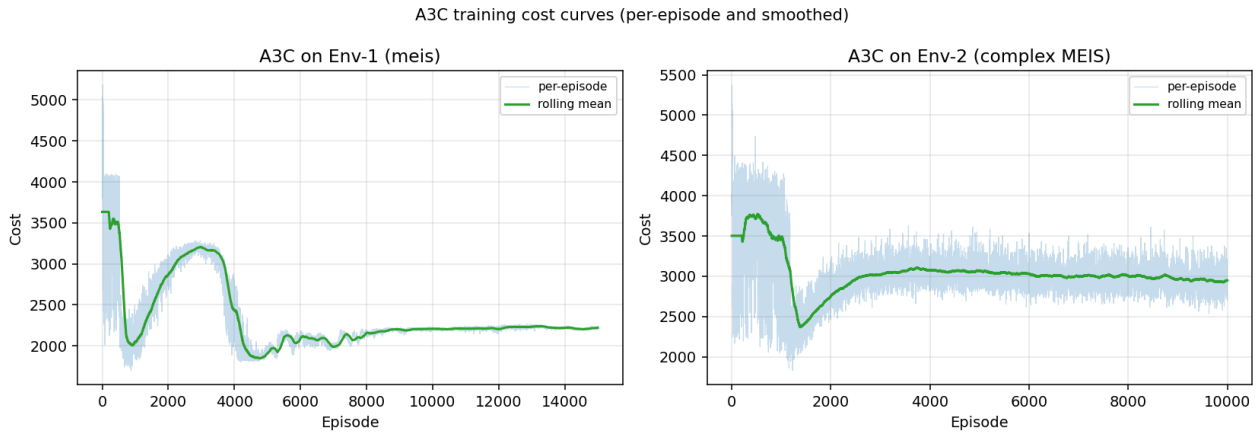

Figure 2: A3C training cost on Env-1 (left) and Env-2 (right). The Env-1 curve shows a mid-training bump around episodes 1500–4500 (policy / entropy oscillation) that ultimately resolves to a stable minimum. On Env-2, convergence settles on a higher cost plateau that, as Section 5.5 will show, is still worse than a retuned heuristic.

Table 3: Headline evaluation results (seed 42). Δ is the relative mean-cost improvement of RL over baseline (positive = RL better). "SL" is service level, which only the MEIS environment exposes.

Algo.	Env	RL mean cost	Baseline mean cost	Δ	SL (RL / BL)
PPO	Env-1	$2{,}600{,}979.87 \pm 29{,}021.34$	$4{,}112{,}775.56 \pm 8{,}759.19$	$+36.76\%$	n/a
PPO	Env-2	$631{,}129.99 \pm 1{,}011{,}936.85$	$10{,}873{,}502.07 \pm 62{,}357.06$	$+94.20\%$	n/a
A3C	Env-1	$1{,}930.24 \pm 4.02$	$1{,}962.40 \pm 52.39$	$+1.64\%$	$95.69\%/95.44\%$
A3C	Env-2	$2{,}881.98 \pm 146.46$	$2{,}366.29 \pm 157.67$	-21.79%	$93.27\%/94.11\%$

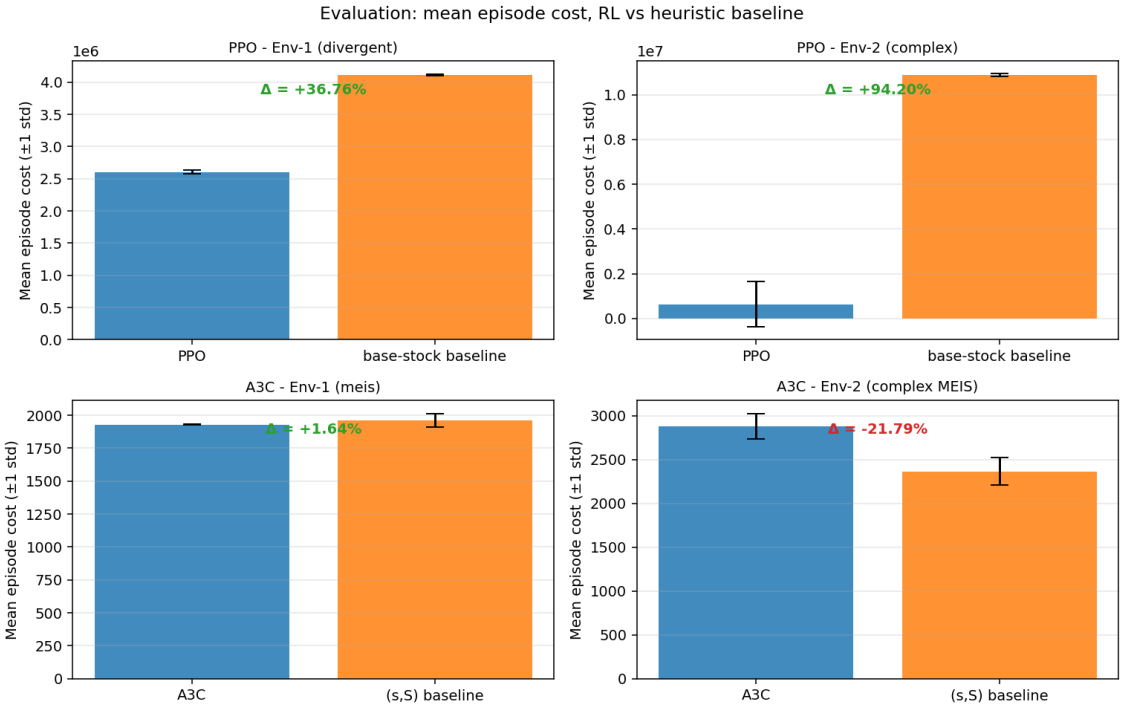

Figure 3: Mean episode cost (lower is better). Error bars are one standard deviation across evaluation episodes. Green = RL wins; red = RL loses. Absolute cost scales differ across algorithms (different cost coefficients and episode lengths); only the within-cell RL vs. baseline gap is interpretable.

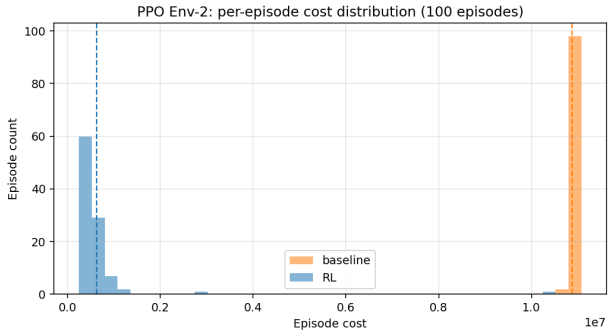

Figure 4: PPO Env-2 per-episode cost distribution (100 episodes). The bulk of PPO episodes (blue) concentrates well below the baseline mode (orange), but a clear right tail of episodes sit close to the baseline mode — these are shock episodes PPO has not learned to hedge.

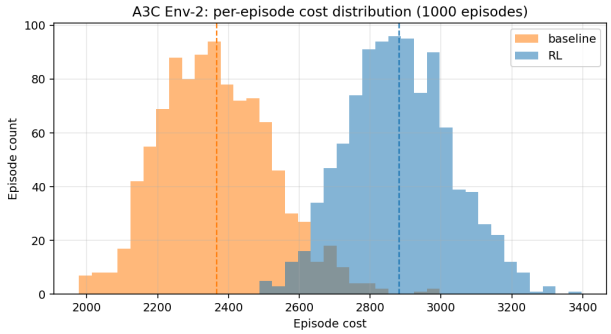

Figure 5: A3C Env-2 per-episode cost distribution (1000 episodes). The baseline (s, S) distribution (orange) lies almost entirely to the left of the A3C distribution (blue): at this training budget, A3C is stochastically dominated by the retuned heuristic.

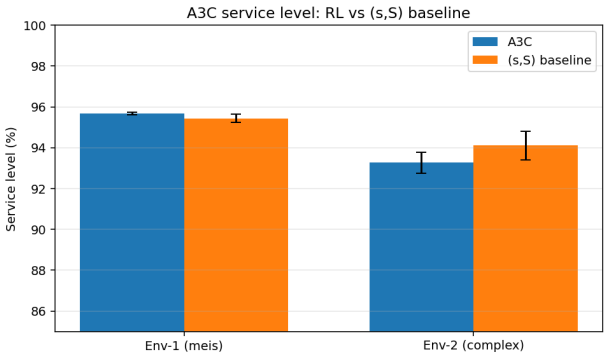

Figure 6: A3C service level on Env-1 and Env-2. On Env-2, A3C is worse than the retuned (s, S) on service level too, mirroring the cost result.

2. *Baseline symmetry matters.* The sign of any RL-vs-heuristic comparison depends on how hard the heuristic is allowed to work. A (s, S) that is retuned per environment is not the same evaluation target as a fixed analytical base-stock, and these two choices end up producing different stories even for identical RL agents.

If we picked the project up again we would (i) rerun all four cells with five seeds and report confidence intervals on the deltas, (ii) give A3C a continuous-action head together with a timestep feature and a recurrent layer so it can actually represent non-stationarity, and (iii) tune the PPO hyperparameters on Env-2 in their own right to get a fairer per-environment comparison.

Acronyms

A3C Asynchronous Advantage Actor-Critic

CI Confidence Interval

DRL Deep Reinforcement Learning

GAE Generalized Advantage Estimation

IP Inventory Position

MDP Markov Decision Process

MEIS Multi-Echelon Inventory System

MLP Multi-Layer Perceptron

OR Operations Research

PPO Proximal Policy Optimization

RL Reinforcement Learning

(s, S) Reorder-point / order-up-to policy

SL Service Level

References

Clark, A. J.; and Scarf, H. 1960. Optimal Policies for a Multi-Echelon Inventory Problem. *Management Science*, 6(4): 475–490.

Cobbe, K.; Hesse, C.; Hilton, J.; and Schulman, J. 2020. Leveraging Procedural Generation to Benchmark Reinforcement Learning. In *ICML*.

Geevers, K.; Van Hezewijk, L.; and Mes, M. R. K. 2024. Multi-Echelon Inventory Optimization Using Deep Reinforcement Learning. *Central European Journal of Operations Research*, 32(3): 653–683.

Gijsbrechts, J.; Boute, R. N.; Van Mieghem, J.; and Zhang, D. 2022. Can Deep Reinforcement Learning Improve Inventory Management? Performance on Lost Sales, Dual-Sourcing, and Multi-Echelon Problems. *Manufacturing & Service Operations Management*, 24(3).

Grahl, J.; Minner, S.; and Dittmar, D. 2016. Meta-Heuristics for Placing Strategic Safety Stock in Multi-Echelon Inventory with Differentiated Service Times. *Annals of Operations Research*, 242(2): 489–504.

Hammler, P.; Riesterer, N.; Mu, G.; and Braun, T. 2022. Multi-Echelon Inventory Optimization Using Deep Reinforcement Learning. In *Quantitative Models in Life Science Business: From Value Creation to Business Processes*, 73–93. Springer.

Lu, X.; Wang, H.; Peng, Z.; Liao, C.; and Liu, C. 2025. Dynamic Optimization of Multi-Echelon Supply Chain Inventory Policies Under Disruptive Scenarios: A Deep Reinforcement Learning Approach. *Symmetry*, 17(12): 2078.

Mnih, V.; et al. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *ICML*.

Oroojlooyjadid, A.; Nazari, M.; Snyder, L. V.; and Takáč, M. 2022. A Deep Q-Network for the Beer Game: Deep Reinforcement Learning for Inventory Optimization. *Manufacturing & Service Operations Management*, 24(1).

Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; and Abbeel, P. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *ICLR*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

Sutton, R. S.; and Barto, A. G. 2018. Temporal-Difference Learning. In *Reinforcement Learning: An Introduction*, 2nd ed., 131–132. MIT Press.

Unyimadu, S. O.; and Anyibuofu, K. A. 2014. Inventory Management Practices in Manufacturing Firms. *Industrial Engineering Letters*, 4(9): 40–44.